
PYTHON
FUNDAMENTOS & LÓGICA
ALGORÍTMICA • POO • CÓDIGO
Autor
ZEDMOUS

ZEDMOUS.DEV

INGENIERÍA DE SOFTWARE, ALGORÍTMICA & LENGUAJES DE PROGRAMACIÓN

GUÍA MAESTRA DE ESTUDIO: FUNDAMENTOS DE PYTHON, ESTRUCTURAS DE DATOS Y POO

AUTORÍA & CÁTEDRA: ING. EDUARDO NIEVES (ZEDMOUS) • SERIE PROGRAMACIÓN (PY-01) – UPP IUJO

UNIDAD 1. INTRODUCCIÓN A LA PROGRAMACIÓN, LÓGICA ALGORÍTMICA Y CICLO E → P → S

FUNDAMENTOS & TIPADO DINÁMICO

¿Qué es Programar y Qué es la Lógica de Programación?

PENSAMIENTO ESTRUCTURADO

Programar consiste en traducir pasos lógicos y no ambiguos a un lenguaje ejecutable por el computador. Implica desarrollar **pensamiento estructurado** y capacidad de **abstracción** para descomponer problemas complejos en partes simples.

La Lógica de Programación nos enseña a pensar de forma ordenada:

- **1.** Identificar qué datos entran (**Entrada**), cómo se operan (**Proceso**) y qué se muestra (**Salida**).
- **2.** Evaluar distintas soluciones y anticipar posibles errores o casos borde.
- **3.** Tomar decisiones condicionales (`if/else`) y repetir acciones (`while/for`).

¿Qué Problemas Resuelve la Programación en la Realidad?

5 DOMINIOS DE IMPACTO

1. **Cálculos matemáticos:** Desde aritmética hasta ecuaciones complejas y física.
2. **Gestión financiera y contable:** Presupuestos, intereses, nóminas y facturación.
3. **Análisis estadístico y datos:** Procesar grandes volúmenes y visualizar patrones.
4. **Automatización de procesos:** Eliminar tareas manuales repetitivas (reportes/scripts).
5. **Desarrollo de software e IA:** Web, apps móviles, videojuegos y modelos de IA.

1. ENTRADA (INPUT) → 2. PROCESO (CEREBRO)

Entrada: Muestra siempre un mensaje claro indicando qué dato esperas. Recuerda que `input()` en Python **siempre devuelve texto (`str`)**: para operar números aplica *casting* con `int()` o `float()`.

Proceso: Es el cerebro del programa (fórmulas, comparaciones, bucles y funciones).



AVATAR: CENTRO

3. SALIDA (PRINT / F-STRINGS) & CONSEJO DE CÁTEDRA

Salida: Comunica el resultado con claridad usando **f-strings**: `print(f"Resultado: {valor}")`.

Regla de Oro: *"Antes de programar, piensa primero en el PROCESO: ¿qué tienes de entrada y qué transformación exacta produce la salida?"*

Tipo Python	Descripción Técnica	Ejemplo Literal
`int`	Números enteros (positivos, negativos o cero)	`42, -7, 0`
`float`	Números reales con punto decimal (IEEE 754)	`3.14, -0.001, 2.0`
`str`	Cadenas de caracteres Unicode inmutables	`"hola", 'Python'`
`bool`	Valores lógicos booleanos (`1` y `0`)	`True, False`
`NoneType`	Ausencia intencional de valor o retorno nulo	`None`
`list`	Colección ordenada y **mutable** de elementos	`[1, 2, 3], ["a", "b"]`
`tuple`	Colección ordenada e **inmutable** de elementos	`(1, 2), ("x", "y")`
`dict`	Mapa de pares `clave: valor` de acceso rápido	`{"nombre": "Ana", "edad": 25}`
`set`	Conjunto no ordenado de elementos **únicos**	`{1, 2, 3}, {"rojo", "azul"}`
`complex`	Números complejos con parte real e imaginaria	`2 + 3j, -1j`
`bytes`	Secuencia inmutable de bytes crudos	`b"hola"`

Casting	Convierte a... / Entradas Válidas	Ejemplo y Resultado
`int(x)`	Entero (desde `float` truncando, `bool` o `str` entero)	`int("42"), int(3.9), int(True)` → `42, 3, 1`
`float(x)`	Decimal (desde `int`, `bool` o `str` numérico)	`float("3.14"), float(5)` → `3.14, 5.0`
`str(x)`	Cadena de texto (acepta cualquier objeto)	`str(123), str(True)` → `"123", "True"`
`bool(x)`	`0, 0.0, "", [], (), {}, None` son `False`; resto `True`	`bool(""), bool(1), bool([])` → `False, True, False`
`list(x)`	Lista mutable (desde iterable `str, tuple, set, range`)	`list("hola")` → `['h', 'o', 'l', 'a']`
`tuple(x)`	Tupla inmutable (desde cualquier iterable)	`tuple([1, 2, 3])` → `(1, 2, 3)`
`set(x)`	Conjunto único (elimina duplicados al instante)	`set([1, 2, 2, 3])` → `{1, 2, 3}`
`dict(x)`	Diccionario (desde pares `(clave, valor)`)	`dict([("a", 1), ("b", 2)])` → `{'a': 1, 'b': 2}`

Advertencia (`ValueError`): Ejecutar `int("hola")` o `int("3.14")` lanza `ValueError: invalid literal for int()`.

ciclo_eps_edad_futura.py — Entrada, Proceso y Salida con f-strings

```python
# 1. ENTRADA: Captura con input() y conversión explícita
nombre = input("Por favor, ingresa tu nombre: ")
edad   = int(input("Ingresa tu edad actual: "))
altura = float(input("Ingresa tu altura en metros: "))
# 2. PROCESO: Transformación matemática (Edad futura y cuadrado)
edad_futura = edad + 5; cuadrado = edad * edad
# 3. SALIDA: Formato interpolado con f-strings
print(f"Hola {nombre} ({altura}m), en 5 años tendrás {edad_futura} años.")
```

adaptacion_proceso.py — Cómo se Adapta el Proceso según el Problema

```python
# Caso A: Proceso aritmético (Sumar dos números)
a = int(input("Núm 1: ")); b = int(input("Núm 2: "))
print(f"La suma es: {a + b}")
# Caso B: Proceso condicional (Aprobación en escala 0-20)
nota = float(input("Ingresa tu nota: "))
if nota >= 10: resultado = "Aprobado"
else:          resultado = "Reprobado"
print(f"Estado: {resultado}")
```

UNIDAD 2. OPERADORES EN PYTHON, CONDICIONES COMPUESTAS Y TABLAS DE LA VERDAD

ÁLGEBRA BOOLEANA & PRECEDENCIA

1. Operadores Aritméticos (Jerarquía PEMDAS: `() → ** → *,/,//,% → +,-`)

Op.	Operación	Ejemplo	Resultado y Detalle Técnico
`+`	Suma	`10 + 4`	`14` (también concatena `str` y `list`)
`-`	Resta	`10 - 4`	`6`
`*`	Multiplicación	`10 * 4`	`40` (repite secuencias: `"Ja" * 3 → "JaJaJa"`)
`/`	División real	`10 / 4`	`2.5` (**siempre** retorna `float` en Python 3)
`//`	División entera	`10 // 4`	`2` (trunca la parte fraccionaria hacia -∞)
`%`	Módulo (Residuo)	`10 % 3`	`1` (clave para detectar paridad: `n % 2 == 0`)
`**`	Potenciación	`2 ** 3`	`8` (2^3; raíz cuadrada: `x ** 0.5`)

2. Operadores de Asignación y Actualización en Sitio

Op.	Descripción	Ejemplo	Equivalencia Exacta
`=`	Asignación simple	`x = 5`	Asigna la referencia `5` a `x`
`+=`	Suma y asigna	`x += 3`	`x = x + 3` (acumulador)
`-=`	Resta y asigna	`x -= 2`	`x = x - 2` (decrementador)
`*=`	Multiplica y asigna	`x *= 4`	`x = x * 4` (productoria)
`/=`	Divide y asigna	`x /= 2`	`x = x / 2` (convierte a `float`)
`//= %= **=`	Entera, módulo, pot.	`x //= 2`	`x = x // 2`, `x = x % 2`, `x = x ** 2`

3. Operadores de Comparación / Relacionales (Retornan `bool`)

Op.	Significado	Ej. (`x=5`)	Evaluación
`==`	Igual a (valor)	`x == 5`	`True` (¡No confundir con `=`!)
`!=`	Diferente de	`x != 3`	`True`
`>`	Mayor estricto que	`x > 2`	`True`
`<`	Menor estricto que	`x < 10`	`True`
`>=`	Mayor o igual que	`x >= 5`	`True`
`<=`	Menor o igual que	`x <= 4`	`False`

4. Operadores Lógicos (Prioridad: `not → and → or`)

Op.	Conectivo	Regla de Verdad	Ejemplo en Python
`and`	Conjunción (∧)	`True` solo si **ambas** son verdaderas	`x > 2 and x < 10` → `True`
`or`	Disyunción (∨)	`True` si **al menos una** es verdadera	`x < 2 or x == 5` → `True`
`not`	Negación (¬)	Invierte el valor booleano	`not (x == 5)` → `False`

CONDICIONES SIMPLES VS. COMPUESTAS Y AGRUPACIÓN CON PARÉNTESIS

- **Condición Simple:** Evalúa una única comparación directa (`if edad >= 18:`) o negada (`if not edad < 18:`).
- **Condición Compuesta:** Combina dos o más proposiciones mediante `and`, `or` y `not`. Cuando mezcles varios conectivos lógicos, **agrupa siempre con paréntesis** `(...)` para controlar el orden exacto de evaluación y evitar ambigüedades lógicas.

```python
# Condición compuesta agrupada con paréntesis de prioridad
edad = 20; tiene_permiso = True; es_estudiante = False
if (edad >= 18 and tiene_permiso) or (es_estudiante and not tiene_permiso):
    print("Acceso permitido al laboratorio")  # Primer bloque es True → Entra
else:
    print("Acceso denegado")
```



AVATAR: DERECHA

2.2 VALORES LÓGICOS Y TABLAS DE LA VERDAD (FUNDAMENTO MATEMÁTICO DEL FLUJO DE CONTROL)

LÓGICA BOOLEANA EN PYTHON

1. Negación (`not A` / ¬A)

INVERSOR UNARIO

Invierte el estado de verdad de la proposición:

A	not A
True	False
False	True

```python
a = True
print(not a)        # False
print(not (5 > 10)) # True
```

2. Conjunción (`A and B` / A ∧ B)

EXIGE AMBAS TRUE

Solo devuelve `True` cuando **ambas** son `True`:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

```python
a = True; b = False
print(a and b)      # False
```

3. Disyunción (`A or B` / A ∨ B)

BASTA UNA TRUE

Solo es `False` cuando **ambas** son `False`:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

```python
a = False; b = True
print(a or b)       # True
```

Refuerzo de Ingeniería — Evaluación por Cortocircuito (*Short-Circuit*) y Comparaciones Encadenadas: En `A and B`, si `A` es `False`, Python **no evalúa** `B` (protege divisiones: `if b != 0 and (a / b) > 2:`). Además, Python permite encadenar desigualdades matemáticas: `if 15 <= temperatura <= 25:`.

UNIDAD 3. ESTRUCTURAS CONDICIONALES (if, elif, else) Y LABORATORIO DE 5 EJERCICIOS

TOMA DE DECISIONES & INDENTACIÓN

Anatomía de `if`, `elif` y `else` en Python

REGLAS SINTÁCTICAS

- **Evaluación secuencial excluyente:** Python evalúa las condiciones de arriba hacia abajo; al encontrar la **primera** condición `True`, ejecuta su bloque e ignora el resto de la escalera.
- **Dos puntos (`:`) e Indentación obligatoria:** Toda cabecera condicional termina con `:` y el bloque interno debe sangrarse con **4 espacios** (estándar PEP 8).

Estructura	¿Cuándo utilizarla?
`if`	Validar una única condición independiente sin camino alternativo.
`if ... else`	Bifurcación binaria: elegir entre dos caminos mutuamente excluyentes.
`if ... elif ... else`	Evaluar múltiples escenarios jerárquicos (3 o más casos).
`x if C else y`	Operador ternario en una línea: `"Par" if n % 2 == 0 else "Impar"`.

escalera_condicional_dias_clima.py — Ejemplos Base de la Guía

```python
# Ejemplo 1: Días de la semana con if - elif - else
dia = "miércoles"
if dia == "lunes":
    print("Inicio de semana")
elif dia == "miércoles":
    print("Mitad de semana")
elif dia == "viernes":
    print("Fin de semana laboral")
else:
    print("Día común")

# Ejemplo 2: Expresión Ternaria en una sola línea
nota = 14
estado = "Aprobado" if nota >= 10 else "Reprobado"
```



AVATAR: IZQUIERDA

TRAMPA CLÁSICA DE ORDEN EN RANGOS NUMÉRICOS (EJERCICIO DE TEMPERATURA)

Cuando evalúes umbrales crecientes con `>` o `>=` (como temperatura o calificaciones), **ordena siempre del valor más alto al más bajo** (ej. primero `> 35`, luego `>= 26`, luego `>= 15`). Si pusieras `if temp >= 15:` de primero, una temperatura de `40°C` entraría en el primer bloque y jamás llegaría a evaluar `> 35`.

3.2 LABORATORIO RESUELTO: LOS 5 EJERCICIOS PRÁCTICOS DE CONDICIONALES

CÓDIGO COMPLETO & VALIDACIÓN

Ejercicios 1 y 2 — Mayoría de Edad y Clasificación de Nota (0 a 20)

```python
# EJERCICIO 1: Verificar si una persona es mayor de edad (>= 18)
edad = int(input("Ingresa tu edad: "))
if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")

# EJERCICIO 2: Clasificar una nota del 0 al 20 (>= 10 Aprobado)
nota = float(input("Ingresa tu nota (0 a 20): "))
if nota >= 10:
    print("Aprobado")
else:
    print("Aplazado")
```

Ejercicios 3 y 4 — Escala Térmica (°C) y Signo de un Número

```python
# EJERCICIO 3: Clasificación por rangos de temperatura en °C
temperatura = float(input("Ingresa la temperatura en °C: "))
if temperatura > 35:
    print("Calor extremo")
elif temperatura >= 26:
    print("Hace calor")
elif temperatura >= 15:
    print("Clima agradable")
else:
    print("Hace frío")

# EJERCICIO 4: Determinar si un número es positivo, negativo o cero
numero = float(input("Ingresa un número: "))
if numero > 0:
    print("El número es positivo")
elif numero < 0:
    print("El número es negativo")
else:
    print("El número es cero")
```

Ejercicio 5 — Calculadora Aritmética Básica con Protección de División entre Cero

```python
# EJERCICIO 5: Pedir dos números y una operación (+, -, *, /) validando b != 0
a = float(input("Ingresa el primer número: "))
b = float(input("Ingresa el segundo número: "))
operacion = input("Ingresa la operación (+, -, *, /): ")

if operacion == "+":
    print(f"Resultado: {a + b}")
elif operacion == "-":
    print(f"Resultado: {a - b}")
elif operacion == "*":
    print(f"Resultado: {a * b}")
elif operacion == "/":
    if b != 0:
        print(f"Resultado: {a / b}")
    else:
        print("Error: división entre cero")
else:
    print("Operación no válida")
```

UNIDAD 4. ESTRUCTURAS REPETITIVAS (BUCLES while Y for) Y LABORATORIO DE 7 EJERCICIOS

ITERACIÓN DETERMINADA E INDETERMINADA

1. Bucle `while` (Por Condición)

CENTINELA / VALIDACIÓN

Repite un bloque **mientras** una condición sea `True`. Como un despertador que vuelve a sonar hasta que decides levantarte. Exige actualizar la variable de control dentro del ciclo para evitar un **bucle infinito**.

```
contador = 1
while contador <= 5:
    print(f"Iteración {contador}")
    contador += 1
```

2. Bucle `for` + `range()`

CONTEO EXACTO

Ideal cuando conoces de antemano el número de repeticiones. `range(inicio, fin, paso)` genera la secuencia entera desde `inicio` hasta `fin - 1` (el límite superior se excluye).

```
# Imprime del 1 al 5 (6 se excluye)
for i in range(1, 6):
    print(f"Iteración {i}")
```

3. `for` sobre Colecciones

RECORRIDO DIRECTO

Recorre directamente cada elemento de un iterable (`list`, `str`, `tuple`, `dict`, `set`). Con `break` detienes el ciclo anticipadamente y con `continue` saltas a la siguiente iteración.

```
colores = ["rojo", "verde", "azul"]
for color in colores:
    print(f"Color: {color}")
```

REPETICIÓN INDETERMINADA VS. DETERMINADA (CASOS REALES DE INGENIERÍA)

- **Repetición Indeterminada:** No sabemos cuántas veces se ejecutará antes de empezar (ej. *procesar facturas en una tienda mientras sigan llegando clientes*). Se modela naturalmente con `while`, aunque un `for` con `break` también puede detenerse por condición.
- **Repetición Determinada:** Tiene un límite fijo conocido antes de iniciar (ej. *un médico que atiende exactamente 10 citas diarias* o el *sistema SAIME programado para un tope máximo de 50 citas diarias*). Se modela con `for`, pudiendo terminar antes si se agota la cola.

Tipo de Repetición	¿Se conoce el límite previo?	Estructura Asociada	¿Puede cambiar en ejecución?
Determinada	Sí (tope fijo, ej. 10 o 50 citas)	`for` (o `while` con contador fijo)	Sí, si se interrumpe con `break`
Indeterminada	No (depende de evento/usuario)	`while` (o `for` sobre flujo dinámico)	Sí, según condiciones externas



AVATAR: DERECHA

4.3 LABORATORIO RESUELTO: LOS 7 EJERCICIOS PRÁCTICOS DE BUCLES (3 CON while + 4 CON for)

ACUMULADORES, CENTINELAS & SERIES

Ejercicios 1, 2 y 3 con while — Contador, Centinela 0 y Contraseña

```
# EJERCICIO 1 (while): Contador del 1 al 10
contador = 1
while contador <= 10:
    print(contador)
    contador += 1

# EJERCICIO 2 (while): Sumar números hasta que el usuario ingrese
0
suma = 0
numero = int(input("Ingresa un número (0 para salir): "))
while numero != 0:
    suma += numero
    numero = int(input("Ingresa otro número (0 para salir): "))
print("Suma total:", suma)

# EJERCICIO 3 (while): Validación de contraseña hasta acertar
clave_correcta = "python123"
clave = input("Ingresa la contraseña: ")
while clave != clave_correcta:
    print("Contraseña incorrecta.")
    clave = input("Intenta de nuevo: ")
print("Acceso concedido.")
```

Ejercicios 4, 5, 6 y 7 con for — Múltiplos, Strings, Listas y Tablas

```
# EJERCICIO 4 (for): Múltiplos de 5 del 5 al 50 con paso 5
for i in range(5, 51, 5):
    print(i)

# EJERCICIO 5 (for): Mostrar cada letra de una palabra
palabra = input("Ingresa una palabra: ")
for letra in palabra:
    print(letra)

# EJERCICIO 6 (for): Sumar los elementos de una lista
numeros = [4, 7, 1, 3, 9]
suma = 0
for n in numeros:
    suma += n
print("Suma total:", suma)  # 24

# EJERCICIO 7 (for): Tabla de multiplicar del 1 al 10
numero = int(input("¿Qué tabla deseas ver?: "))
for i in range(1, 11):
    print(f"{numero} x {i} = {numero * i}")
```

UNIDAD 5. FUNCIONES MODULARES, LAMBDA, *args / **kwargs, RECURSIÓN Y CATÁLOGO BUILT-IN

ARQUITECTURA MODULAR & REUTILIZACIÓN

tipos_funciones.py — Las 4 Tipologías Clásicas + Condicionales y Ciclos

```python
# 1. Sin parámetros y sin retorno
def saludar():
    print("¡Hola, bienvenido al curso!")

# 2. Con parámetros y sin retorno
def mostrar_nombre(nombre):
    print(f"Hola, {nombre}")

# 3. Sin parámetros y con retorno
def obtener_pi():
    return 3.1416

# 4. Con parámetros, retorno, Docstring y Type Hints (PEP 484)
def sumar(a: int, b: int) -> int:
    """Suma dos números enteros y devuelve el resultado."""
    return a + b

# 5. Función con condicional y función con ciclo
def verificar_par(numero):
    return "Par" if numero % 2 == 0 else "Impar"

def imprimir_lista(lista):
    for elemento in lista:
        print(elemento)
```

funciones_avanzadas.py — Anidadas, Lambda, Orden Superior, *args, **kwargs, Recursión

```python
# 6. Función anidada y 7. Función Lambda (anónima en una línea)
def operacion():
    def sumar_interno(x, y): return x + y
    return sumar_interno(3, 4)

cuadrado = lambda x: x ** 2  # cuadrado(5) → 25

# 8. Función de Orden Superior (recibe otra función como parámetro)
def aplicar_operacion(func, a, b): return func(a, b)
def multiplicar(x, y): return x * y
print(aplicar_operacion(multiplicar, 5, 6))  # 30

# 9. Parámetro opcional, 10. *args (tupla) y **kwargs (diccionario)
def saludar_opt(nombre="visitante"): print(f"Hola, {nombre}")
def sumar_todo(*numeros): return sum(numeros)  # sumar_todo(1,2,3,4) → 10
def mostrar_info(**persona):
    for k, v in persona.items(): print(f"{k}: {v}")

# 11. Función Recursiva (Caso base n == 1 + llamada recursiva)
def factorial(n):
    if n <= 1: return 1
    return n * factorial(n - 1)  # factorial(5) → 120
```



AVATAR: IZQUIERDA

BUENAS PRÁCTICAS DE INGENIERÍA AL DISEÑAR FUNCIONES EN PYTHON

1. **Responsabilidad Única:** Cada función debe hacer una sola tarea bien definida y su nombre en `snake_case` debe describir su acción (ej. `calcular_impuesto()`).
2. `return` **vs** `print()`**:** Prefiere retornar valores con `return` en lugar de solo imprimirlos; así el resultado puede encadenarse en otras expresiones o probarse con tests unitarios.
3. **Documentación:** Incluye siempre un *docstring* `"""..."""` en la primera línea explicando entradas y salidas.

5.3 CATÁLOGO CLASIFICADO DE LAS 43 FUNCIONES PREDEFINIDAS (BUILT-IN FUNCTIONS DE PYTHON)

DISPONIBLES SIN IMPORTAR MÓDULOS

Categoría	Funciones Built-in	Propósito Técnico y Ejemplos Directos
1. Conversión de Tipos (12)	`int()` `float()` `str()` `bool()` `list()` `tuple()` `set()` `dict()` `complex()` `bin()` `oct()` `hex()`	Transforman tipos primitivos y colecciones. Además, convierten enteros a representación binaria `bin(10) → '0b1010'`, octal `oct(10) → '0o12'` y hexadecimal `hex(255) → '0xff'`.
2. Entrada y Salida (2)	`print()` `input()`	`print(*objetos, sep=' ', end='\n')` muestra datos en consola; `input(prompt)` lee una línea del usuario como `str`.
3. Conteo, Orden y Agregación (6)	`len()` `sum()` `max()` `min()` `sorted()` `reversed()`	`len(x)` longitud; `sum(x)` suma total; `max(x)/min(x)` extremos; `sorted(x, reverse=False)` devuelve nueva lista ordenada; `reversed(x)` iterador inverso.
4. Evaluación y Tipos (5)	`type()` `isinstance()` `eval()` `exec()` `id()`	`type(x)` retorna la clase; `isinstance(5, int) → True` verifica herencia/tipo seguro; `id(x)` dirección única en memoria; `eval()/exec()` evalúan código dinámico en texto.
5. Matemáticas y Numéricas (4)	`abs()` `round()` `pow()` `divmod()`	`abs(-7) → 7` valor absoluto; `round(3.14159, 2) → 3.14` redondeo; `pow(2, 3) → 8` potencia; `divmod(10, 3) → (3, 1)` retorna tupla `(cociente, residuo)`.
6. Operación sobre Iterables (7)	`range()` `enumerate()` `zip()` `map()` `filter()` `all()` `any()`	`enumerate(it)` entrega pares `(índice, valor)`; `zip(a, b)` empareja múltiples iterables; `map(fn, it)` transforma cada ítem; `filter(fn, it)` filtra por predicado; `all(it)` / `any(it)` cuantificadores lógicos (∀ y ∃).
7. Verificación y Errores (1)	`assert`	`assert condicion, "Mensaje"` verifica invariantes de depuración; si la condición es `False`, lanza `AssertionError`.
8. Introspección y Clases (6)	`dir()` `help()` `callable()` `globals()` `locals()` `vars()`	`dir(obj)` lista atributos/métodos; `help(obj)` abre el manual interactivo; `callable(obj)` indica si se puede invocar con `()`; `vars(obj)` retorna el diccionario `__dict__` del objeto.

UNIDAD 6. ESTRUCTURAS DE DATOS EN PYTHON: COLECCIONES, PILAS, COLAS Y COMPRENSIÓN

ALMACENAMIENTO & COMPLEJIDAD

1. Listas (`list`) — Ordenadas, Mutables y con Duplicados

INDEXACIÓN 0..N-1

Secuencia dinámica ideal cuando los datos deben modificarse, ordenarse o crecer en ejecución: `frutas = ["manzana", "banana", "pera"]` (acceso: `frutas[0]`, último: `frutas[-1]`).

Método de Lista	Acción en Memoria
`lista.append(x)`	Agrega el elemento `x` al final de la lista (tiempo constante `O(1)`).
`lista.insert(i, x)`	Inserta `x` exactamente en la posición de índice `i`.
`lista.remove(x)`	Elimina la primera aparición del valor `x`.
`lista.pop([i])`	Extrae y retorna el elemento en `i` (por defecto el último).
`lista.sort()`	Ordena la lista *in-place* (menor a mayor o alfabético).
`lista.reverse()`	Invierte el orden de los elementos en el sitio.

2. Diccionarios (`dict`) — Pares `clave: valor`

BÚSQUEDA HASH O(1)

Estructura asociativa donde cada valor se recupera mediante una clave única e inmutable: `persona = {"nombre": "Ana", "edad": 30, "ciudad": "Caracas"}`.

Método de Diccionario	Acción en Memoria
`d.get(clave, def)`	Devuelve el valor de `clave` sin lanzar `KeyError` si no existe.
`d.keys()`	Vista iterable con todas las claves del diccionario.
`d.values()`	Vista iterable con todos los valores almacenados.
`d.items()`	Vista de tuplas `(clave, valor)` ideal para bucles `for k, v`.
`d.update(otro_dict)`	Agrega o sobrescribe múltiples pares `clave: valor`.
`d.pop(clave)`	Elimina la clave indicada y retorna su valor asociado.

3. Tuplas (`tuple`) — Ordenadas e Inmutables

PROTECCIÓN & VELOCIDAD

Una vez creada una tupla, sus elementos **no pueden modificarse** (intentar `coordenadas[0] = 99` lanza `TypeError`). Ventajas clave frente a las listas:

- **Integridad de datos:** Protege constantes, coordenadas `(10, 20)` o registros fijos.
- **Eficiencia:** Consumen menos memoria RAM y su recorrido es más veloz.
- **Desempaquetado elegante:** `x, y = (10, 20)` asigna `x=10` e `y=20` al instante.

4. Conjuntos (`set`) — Elementos Únicos y Álgebra de Conjuntos

SIN DUPLICADOS

Colección no ordenada que descarta duplicados automáticamente: `{1, 2, 2, 3}` → `{1, 2, 3}`. Soporta las operaciones formales de teoría de conjuntos:

- **Unión (A ∪ B):** `a.union(b)` o `a | b` (todos los elementos de ambos).
- **Intersección (A ∩ B):** `a.intersection(b)` o `a & b` (elementos comunes).
- **Diferencia (A – B):** `a.difference(b)` o `a - b` (en A pero no en B).

PILAS (STACK — LIFO: LAST IN, FIRST OUT)

El último elemento en entrar es el primero en salir (como una pila de platos o el historial *Deshacer Ctrl+Z*). En Python se implementa directamente sobre `list` usando `append()` para apilar (*push*) y `pop()` para desapilar.



AVATAR: CENTRO

COLAS (QUEUE — FIFO: FIRST IN, FIRST OUT)

El primero en llegar es el primero en salir (como la fila de atención de un banco o cola de impresión). En Python se usa `collections.deque` porque `popleft()` extrae del frente en tiempo constante `O(1)`.

pilas_y_colas.py — Implementación LIFO (list) y FIFO (collections.deque)

```python
# 1. PILA (LIFO - Last In, First Out) usando list
pila = []
pila.append("plato 1")
pila.append("plato 2")
ultimo = pila.pop()  # Extrae "plato 2" (el último que entró)

# 2. COLA (FIFO - First In, First Out) usando deque
from collections import deque
cola = deque()
cola.append("cliente 1")
cola.append("cliente 2")
primero = cola.popleft()  # Atiende a "cliente 1" en O(1)
```

comprension_listas.py — List Comprehension (Sintaxis Declarativa Compacta)

```python
# Sintaxis general: [expresion for elemento in iterable if condicion]

# Ejemplo 1: Cuadrados de los números del 0 al 4
cuadrados = [x**2 for x in range(5)]
print(cuadrados)  # [0, 1, 4, 9, 16]

# Ejemplo 2: Filtrar únicamente los números pares del 0 al 9
pares = [x for x in range(10) if x % 2 == 0]
print(pares)      # [0, 2, 4, 6, 8]
```

UNIDAD 7. PROGRAMACIÓN ORIENTADA A OBJETOS (POO): CLASES, ENCAPSULACIÓN, HERENCIA Y POLIMORFISMO

MODELADO DE OBJETOS & DUCK TYPING

7.1 ¿Qué es la POO? Clases, Objetos, `__init__` y `self`

MOLDE VS. INSTANCIA

La **Programación Orientada a Objetos (POO)** organiza el software en torno a **objetos** que combinan estado (**atributos**) y comportamiento (**métodos**):

- **Clase (`class`):** Es el molde o plano arquitectónico (ej. `Perro`, `Cuenta`).
- **Objeto (Instancia):** Es la entidad concreta creada en memoria a partir de la clase.
- **Constructor `__init__(self, ...)`:** Método especial que inicializa los atributos automáticamente al instanciar el objeto.
- **Referencia `self`:** Apunta a la instancia actual que invoca el método.

clase_perro.py — Definición de Clase, Constructor e Instanciación

```python
class Perro:
    especie = "Canino"  # Atributo de clase (compartido por todos)

    def __init__(self, nombre, edad):
        self.nombre = nombre  # Atributo de instancia
        self.edad = edad

    def ladrar(self):
        print(f"{self.nombre} ({self.especie}) dice: ¡Guau!")

perro1 = Perro("Fido", 3)
perro1.ladrar()  # Fido (Canino) dice: ¡Guau!
```

7.2 Visibilidad de Atributos y Encapsulación (Python vs. Java)

Nivel	Sintaxis en Python	Equivalente Java	Comportamiento
Público	`self.nombre`	`public`	Accesible libremente desde dentro y fuera de la clase.
Protegido	`self._edad`	`protected`	Convención: usar solo dentro de la clase y sus subclases.
Privado	`self.__clave`	`private`	Activa *Name Mangling* (`_Clase__clave`) ocultándolo del exterior.

encapsulacion_cuenta.py — Protección del Saldo Bancario

```python
class Cuenta:
    def __init__(self, saldo):
        self.__saldo = saldo  # Atributo privado encapsulado

    def depositar(self, monto):
        if monto > 0:
            self.__saldo += monto

    def consultar_saldo(self):
        return self.__saldo
```

tipos_metodos.py — Instancia (self), Clase (@classmethod) y Estático (@staticmethod)

```python
class Ejemplo:
    contador_global = 0

    def metodo_instancia(self):
        # Recibe self: accede a los atributos del objeto individual
        print("Método normal de instancia", self)

    @classmethod
    def metodo_clase(cls):
        # Recibe cls: opera sobre la clase o crea constructores alternativos
        cls.contador_global += 1
        print("Método de clase", cls)

    @staticmethod
    def metodo_estatico():
        # No recibe ni self ni cls: función utilitaria dentro del namespace
        print("Método estático independiente")
```

herencia_polimorfismo.py — Jerarquía de Clases

```python
class Animal:
    def hablar(self):
        print("Hace un sonido")

class Perro(Animal):
    def hablar(self): print("Guau!")

class Gato(Animal):
    def hablar(self): print("Miau!")

for a in [Perro(), Gato()]:
    a.hablar()  # Polimorfismo en acción
```

duck_typing.py — Polimorfismo sin Herencia

```python
class Loro:
    def hablar(self): print("¡Hola!")

class Robot:
    def hablar(self): print("Bip bop")

def hacer_hablar(objeto):
    # Si tiene el método .hablar(), funciona
    objeto.hablar()

hacer_hablar(Loro())
hacer_hablar(Robot())
```



AVATAR: DERECHA

UNIDAD 8. MÓDULOS, ENTORNOS VIRTUALES (venv), EXCEPCIONES, ARCHIVOS Y RECURSOS

INGENIERÍA DE SOFTWARE & PERSISTENCIA

8.1 Módulos (`.py`), Paquetes (`__init__.py`) y Gestor `pip`

ARQUITECTURA DE PROYECTO

- **Módulo:** Todo archivo `.py` reutilizable (ej. `operaciones.py` con `def sumar(a, b): return a + b`) que se importa mediante `import operaciones` o `from operaciones import sumar`.
- **Paquete:** Carpeta que agrupa varios módulos e incluye un archivo `__init__.py`.
- **Gestor `pip`:** Instala librerías externas desde PyPI:

```
# Comandos esenciales de pip en terminal
pip install requests     # Instalar librería externa
pip uninstall requests   # Desinstalar librería
pip list                 # Listar paquetes instalados
```

8.2 Entornos Virtuales (`venv`) y `requirements.txt`

AISLAMIENTO DE DEPENDENCIAS

Un entorno virtual aísla las librerías de cada proyecto para evitar conflictos de versiones con el sistema operativo:

```
# 1. Crear el entorno virtual en la carpeta del proyecto
python -m venv .venv

# 2. Activar en Windows (PowerShell / CMD) vs Linux / macOS
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux / macOS

# 3. Exportar e instalar dependencias reproducibles y salir
pip freeze > requirements.txt   # Congelar versiones
pip install -r requirements.txt # Replicar en otro equipo
deactivate                      # Desactivar entorno
```

8.3 Manejo de Excepciones (try / except / else / finally)

```
# Captura controlada de errores para que el programa no colapse
try:
    x = int(input("Ingresa un divisor entero: "))
    resultado = 10 / x
except ZeroDivisionError:
    print("Error: No se puede dividir entre cero.")
except ValueError:
    print("Error: Debes ingresar un número válido.")
except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")
else:
    print(f"División exitosa: {resultado}")
finally:
    print("Fin del bloque (se ejecuta siempre).")
```

8.4 Manejo de Archivos con `with open(ruta, modo)`

Modo	Significado	Comportamiento
`"r"`	Lectura (*Read*)	Abre archivo existente: lanza `FileNotFoundError` si no existe.
`"w"`	Escritura (*Write*)	Crea el archivo o **sobrescribe** todo su contenido previo.
`"a"`	Anexar (*Append*)	Agrega líneas nuevas al final sin borrar lo existente.
`"x"` / `"b"`	Exclusivo / Binario	`"x"` falla si ya existe; `"b"` lee/escribe bytes (imágenes/PDF).

```
# El administrador de contexto "with" cierra el archivo automáticamente
with open("datos.txt", "w", encoding="utf-8") as f:
    f.write("Hola, mundo\nSegunda línea\n")

try:
    with open("datos.txt", "r", encoding="utf-8") as f:
        for linea in f:
            print(linea.strip())  # .strip() quita el salto \n
except FileNotFoundError:
    print("El archivo no existe.")
```



AVATAR: IZQUIERDA

PLATAFORMAS DE ENTRENAMIENTO ALGORÍTMICO Y REPOSITORIO OFICIAL DEL CURSO

La programación no se domina solo leyendo: se domina resolviendo retos reales en consola. Aquí tienes las 3 plataformas recomendadas en clase y el repositorio oficial de GitHub con todo el código práctico visto en el curso:

1. Boot.dev
Ruta Backend Python:
`boot.dev`

2. HackerRank
Retos y certificación:
`hackerrank.com`

3. LeetCode
Algoritmos y entrevistas:
`leetcode.com`

4. Repo Oficial GitHub
Código y laboratorios:
`github.com/Zedmous/zmpython`